TSwap Protocol Audit Report

Version 1.0

Cyfrin.io

September 5, 2025

TSwap Audit Report

thecil.

September 5, 2025

Prepared by: thecil Lead Auditors: - thecil - Carlos Zambrano.

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
  - Scope
  - Roles
- Executive Summary
  - Issues found
- Findings
- High
- Medium
- Low
- Informational
- Gas

Protocol Summary

This project is meant to be a permissionless way for users to swap assets between each other at a fair price. You can think of T-Swap as a decentralized asset/token exchange (DEX). T-Swap is known as an Automated Market Maker (AMM) because it doesn't use a normal "order book" style exchange, instead it uses "Pools" of an asset. It is similar to Uniswap. To understand Uniswap, please watch this video: Uniswap Explained

Disclaimer

The thecil team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

Scope

```
src/PoolFactory.sol
src/TSwapPool.sol
```

Lines of Code

Filepath	nSLOC
src/PoolFactory.sol	35
src/TSwapPool.sol	317
Total	**352**

Roles

NA

Executive Summary

Issues found

Severity	Number of issues found
High	4
Medium	1
Low	2
Info	12
Total	19

Findings

High

[H-1] - Incorrect fee calculation in `TSwapPool::getInputAmountBasedOnOutput` causes protocol to take too many tokens from user, resultin in lost fees.

Description: The `getInputAmountBasedOnOutput` function is intended to calculate the amount of tokens a user should deposit given an amount of tokens of output tokens. However, the function

currently miscalculates the resulting amount. When calculating the fee, it scales the amount by 10_000 instead of 1_000.

Impact: Protocol takes more fees than expected from users.

Proof of Concept: (Proof of Code)

The following code is the actual codebase of the `TSwapPool::getInputAmountBasedOnOutput` function:

view code

```
    function getInputAmountBasedOnOutput(
        uint256 outputAmount,
        uint256 inputReserves,
        uint256 outputReserves
    )
        public
        pure
        revertIfZero(outputAmount)
        revertIfZero(outputReserves)
        returns (uint256 inputAmount)
    {
        return
@>          ((inputReserves * outputAmount) * 10000) /
            ((outputReserves - outputAmount) * 997);
    }
```

Recommended Mitigation: Change the `10000` magic number to `1000` to harmonize the calculation of the function:

```
    function getInputAmountBasedOnOutput(
        uint256 outputAmount,
        uint256 inputReserves,
        uint256 outputReserves
    )
        public
        pure
        revertIfZero(outputAmount)
        revertIfZero(outputReserves)
        returns (uint256 inputAmount)
    {
        return
-           ((inputReserves * outputAmount) * 10000) /
+           ((inputReserves * outputAmount) * 1000) /
            ((outputReserves - outputAmount) * 997);
    }
```

Also keep into consideration the `[I-9]` rule by replacing the magic number `1000` with a constant value that aligns with the expected precision for this calculations.

[H-2] - Lack of slippage protection in `TSwapPool::swapExactOutput` causes users to potentially receive way fewer tokens.

Description: The `swapExactOutput` function does not include any sort of slippage protection. This function is similar to what is done in `TSwapPool::swapExactInput` where the function specifies a `minOutputAmount`, the `swapExactOutput` function should specify a `maxInputAmount`.

Impact: If market conditions change before the transaction processes, the user could get a much worse swap.

Proof of Concept:

1. The price of 1 WETH right now is 1,000 USDC.
2. User inputs a `swapExactOutput` looking for 1 WETH.

```
inputToken = USDC
outputToken = WETH
outputAmount = 1
deadline = whatever
```

3. The function does not offer a maxInput amount.
4. As the transaction is pending in the mempool, the market changes, price move HUGE, 1 WETH is now 10,0000 USDC, 10x more than the user expected.
5. The transaction completes, but the user sent the protocol 10,0000 USDC instead of the expected 1,000 USDC.

Recommended Mitigation: Refactor the `swapExactOutput` function to include a maximum input amount (`maxInputAmount`) parameter. This ensures that the user cannot submit a transaction for more than the expected output amount, thereby mitigating the risk of receiving a much worse swap.

```
+    error TSwapPool__InputTooLow(uint256 inputedAmount, uint256 maxAmount);

     function swapExactOutput(
         IERC20 inputToken,
         IERC20 outputToken,
         uint256 outputAmount,
+        uint256 maxInputAmount
         uint64 deadline
     )
         public
         revertIfZero(outputAmount)
         revertIfDeadlinePassed(deadline)
         returns (uint256 inputAmount)
     {
         uint256 inputReserves = inputToken.balanceOf(address(this));
```

```
16          uint256 outputReserves = outputToken.balanceOf(address(this));
17
18          inputAmount = getInputAmountBasedOnOutput(
19              outputAmount,
20              inputReserves,
21              outputReserves
22          );
23
24  +       if (inputAmount < maxInputAmount) {
25  +           revert TSwapPool__InputTooLow(inputAmount, maxInputAmount);
26  +       }
27          _swap(inputToken, inputAmount, outputToken, outputAmount);
28      }
```

[H-3] - `TSwapPool::sellPoolTokens` mismatches input and output tokens causing users to receive the incorrect amount of tokens.

Description: The `sellPoolTokens` function is intended to allow users to easely sell pool tokens and receive WETH in exchange. Users indicate how many pool tokens they're willing to sell in the `poolTokenAmount` parameter. However, the function currently miscalculates the swapped amount.

This is due the fact that the `swapExactOutput` function is called, whereas the `swapExactInput` function is the one that should be called. Because users specify the exact amount of input tokens, not output.

Impact: Users will swap the wrong amount of tokens, which is a severe disruption of protocol functionality.

Proof of Concept: (Proof of Code)

Recommended Mitigation: Consider changing the implementation to use `swapExactInput` instead of `swapExactOutput`. Note that this would alsow require changing the `sellPoolTokens` function to accept a new parameter (ie `minWethToReceive` to be passed to `swapExactInput`).

```
1       function sellPoolTokens(
2           uint256 poolTokenAmount
3   +       uint256 minWethToReceive
4           ) external returns (uint256 wethAmount) {
5   -        return swapExactOutput(i_poolToken, i_wethToken,
        poolTokenAmount, uint64(block.timestamp));
6   +        return swapExactInput(i_poolToken, poolTokenAmount,
        i_wethToken, minWethToReceive, uint64(block.timestamp))
7       }
```

Additionally, it might be wise to add a deadline to the function, as there is currently no deadline.

[H-4] - In `TSwapPool::_swap` the extra tokens given to users after every `swapCount` breaks the protocol invarian of `x * y = k`.

Description: The protocol follows a strict invariant of `x * y = k`. Where: - `x`: The balance of the pool token. - `y`: The balance of WETH. - `k`: the constant product of the two balances.

This means, tat whenever the balances change in the protocol, the ratio between the two amounts should remain constant, hence the `k`. However, this is broken due to the extra incentive in the `_swap` function. Meaning that over time the protocol funds will be drained.

Impact: A user could maliciously drain the protocol of funds by doing a lot of swaps and collecting the extra incentive given out by the protocol.

Most simply put, the protocol's core invariant is broken.

The follow block of code in the `TSwapPool::_swap` function, is responsible for the issue:

```
    swap_count++;
    if (swap_count >= SWAP_COUNT_MAX) {
        swap_count = 0;
        outputToken.safeTransfer(msg.sender, 1_000_000_000_000_000_000)
            ;
    }
```

Proof of Concept: 1. A user swaps 10 times, and collects the extra incentive of `1_000_000_000_000_000_000` tokens. 2. The user continues to swap until all the protocol funds are drained.

view code

```
    function test_invariantBroken() public {
        // provide liquidity to the pool
        vm.startPrank(liquidityProvider);
        weth.approve(address(pool), 100e18);
        poolToken.approve(address(pool), 100e18);
        pool.deposit(100e18, 100e18, 100e18, uint64(block.timestamp));
        vm.stopPrank();

        uint256 outputWeth = 1e17;

        vm.startPrank(user);
        poolToken.approve(address(pool), type(uint256).max);
        poolToken.mint(user, 100e18);
        // user will swap 9 times
        for (uint256 i = 0; i < 9; i++) {
            pool.swapExactOutput(
                poolToken,
                weth,
                outputWeth,
                uint64(block.timestamp)
```

```
            );
        }
        int256 startingY = int256(weth.balanceOf(address(pool)));
        int256 expectedDeltaY = int256(-1) * int256(outputWeth);
        // user will swap the 10th time to break the protocol invariant
        pool.swapExactOutput(
            poolToken,
            weth,
            outputWeth,
            uint64(block.timestamp)
        );
        vm.stopPrank();

        uint256 endingY = weth.balanceOf(address(pool));
        int256 actualDeltaY = int256(endingY) - int256(startingY);
        assertEq(actualDeltaY, expectedDeltaY);
    }
```

Recommended Mitigation: Remove the extra incentive mechanism. If you want to keep this in, we should account for the change in the `x * y = k` protocol invariant. Or, we should set aside tokens in the same way we do with fees.

```
-   swap_count++;
-   if (swap_count >= SWAP_COUNT_MAX) {
-       swap_count = 0;
-       outputToken.safeTransfer(msg.sender, 1_000_000_000_000_000_000);
-   }
```

Medium

[M-1] - `TSwapPool::deposit` function `deadline` parameter is missing a check, causing trasactions to complete even after the deadline.

Description: The deposit function in the TSwapPool contract accepts a deadline parameter but does not utilize it to verify whether the transaction has been submitted before the deadline. This omission allows users to submit transactions after the intended time, potentially causing disruptions to the functionality of the system.

Impact: High.

A user who expects a deposit to fail because it exceeds the deadline will be able to proceed with the transaction, leading to severe disruption of the protocol.

Impact: High.

This condition is always true unless explicitly checked and enforced within the function.

Proof of Concept: (Proof of Code)

This is the actual code for the function:

view code

```
    function deposit(
        uint256 wethToDeposit,
        uint256 minimumLiquidityTokensToMint,
        uint256 maximumPoolTokensToDeposit,
        uint64 deadline
    )
        external
        revertIfZero(wethToDeposit)
        returns (uint256 liquidityTokensToMint)
    {
        if (wethToDeposit < MINIMUM_WETH_LIQUIDITY) {
            revert TSwapPool__WethDepositAmountTooLow(
                MINIMUM_WETH_LIQUIDITY,
                wethToDeposit
            );
        }
        if (totalLiquidityTokenSupply() > 0) {
            uint256 wethReserves = i_wethToken.balanceOf(address(this))
                ;
            uint256 poolTokenReserves = i_poolToken.balanceOf(address(
                this));
            // Our invariant says weth, poolTokens, and liquidity
                tokens must always have the same ratio after the
            // initial deposit
            // poolTokens / constant(k) = weth
            // weth / constant(k) = liquidityTokens
            // aka...
            // weth / poolTokens = constant(k)
            // To make sure this holds, we can make sure the new
                balance will match the old balance
            // (wethReserves + wethToDeposit) / (poolTokenReserves +
                poolTokensToDeposit) = constant(k)
            // (wethReserves + wethToDeposit) / (poolTokenReserves +
                poolTokensToDeposit) =
            // (wethReserves / poolTokenReserves)
            //
            // So we can do some elementary math now to figure out
                poolTokensToDeposit...
            // (wethReserves + wethToDeposit) = (poolTokenReserves +
                poolTokensToDeposit) * (wethReserves /
            // poolTokenReserves)
            // wethReserves + wethToDeposit  = poolTokenReserves * (
                wethReserves / poolTokenReserves) +
            // poolTokensToDeposit * (wethReserves / poolTokenReserves)
            // wethReserves + wethToDeposit = wethReserves +
                poolTokensToDeposit * (wethReserves / poolTokenReserves)
```

```
37              // wethToDeposit / (wethReserves / poolTokenReserves) =
                    poolTokensToDeposit
38              // (wethToDeposit * poolTokenReserves) / wethReserves =
                    poolTokensToDeposit
39              uint256 poolTokensToDeposit =
                    getPoolTokensToDepositBasedOnWeth(
40                  wethToDeposit
41              );
42              if (maximumPoolTokensToDeposit < poolTokensToDeposit) {
43                  revert TSwapPool__MaxPoolTokenDepositTooHigh(
44                      maximumPoolTokensToDeposit,
45                      poolTokensToDeposit
46                  );
47              }
48
49              // We do the same thing for liquidity tokens. Similar math.
50              liquidityTokensToMint =
51                  (wethToDeposit * totalLiquidityTokenSupply()) /
52                  wethReserves;
53              if (liquidityTokensToMint < minimumLiquidityTokensToMint) {
54                  revert TSwapPool__MinLiquidityTokensToMintTooLow(
55                      minimumLiquidityTokensToMint,
56                      liquidityTokensToMint
57                  );
58              }
59              _addLiquidityMintAndTransfer(
60                  wethToDeposit,
61                  poolTokensToDeposit,
62                  liquidityTokensToMint
63              );
64          } else {
65              // This will be the "initial" funding of the protocol. We
                    are starting from blank here!
66              // We just have them send the tokens in, and we mint
                    liquidity tokens based on the weth
67              _addLiquidityMintAndTransfer(
68                  wethToDeposit,
69                  maximumPoolTokensToDeposit,
70                  wethToDeposit
71              );
72              liquidityTokensToMint = wethToDeposit;
73          }
74      }
```

Recommended Mitigation: Add the `revertIfDeadlinePassed` modifier to ensure that the transaction has been submitted before the specified deadline.

```
1       function deposit(
2           uint256 wethToDeposit,
3           uint256 minimumLiquidityTokensToMint,
4           uint256 maximumPoolTokensToDeposit,
```

```
        uint64 deadline
    )
        external
+       revertIfDeadlinePassed(deadline)
        revertIfZero(wethToDeposit)
        returns (uint256 liquidityTokensToMint)
```

Low

[L-1] - `TSwapPool::_addLiquidityMintAndTransfer` private function emits the `LiquidityAdded` event with incorrect order of the events parameters.

Description: The `LiquidityAdded` event emmited by the `_addLiquidityMintAndTransfer` function has the parameters in a wrong order.

Impact: Low

Proof of Concept: (Proof of Code)

The following code is the actual codebase of the `TSwapPool::_addLiquidityMintAndTransfer` .

```
    function _addLiquidityMintAndTransfer(
        uint256 wethToDeposit,
        uint256 poolTokensToDeposit,
        uint256 liquidityTokensToMint
    ) private {
        _mint(msg.sender, liquidityTokensToMint);
        emit LiquidityAdded(msg.sender, poolTokensToDeposit,
            wethToDeposit);

        // Interactions
        i_wethToken.safeTransferFrom(msg.sender, address(this),
            wethToDeposit);
        i_poolToken.safeTransferFrom(
            msg.sender,
            address(this),
            poolTokensToDeposit
        );
    }
```

Recommended Mitigation: Modify the `TSwapPool::_addLiquidityMintAndTransfer` function to emit the event with the correct order of parameters.

```

    function _addLiquidityMintAndTransfer(
        uint256 wethToDeposit,
```

```
4           uint256 poolTokensToDeposit,
5           uint256 liquidityTokensToMint
6       ) private {
7           _mint(msg.sender, liquidityTokensToMint);
8   -       emit LiquidityAdded(msg.sender, poolTokensToDeposit,
        wethToDeposit);
9   +       emit LiquidityAdded(msg.sender, wethToDeposit,
        poolTokensToDeposit);
10
11          // Interactions
12          i_wethToken.safeTransferFrom(msg.sender, address(this),
                wethToDeposit);
13          i_poolToken.safeTransferFrom(
14              msg.sender,
15              address(this),
16              poolTokensToDeposit
17          );
18      }
```

[L-2] - Default value returned by `TSwapPool::swapExactInput` function results in incorrect return value given.

Description: The `TSwapPool::swapExactInput` function is expected to return the actual amount of tokens bought by the caller. However, while it declares the named return value `output` it is never assigned a value, nor uses an explict return statement.

Impact: The return value will always be 0, giving incorrect information to the caller.

Proof of Concept: (Proof of Code)

The following unit test demostrates this issue:

```
1       function test_SwapExactAmountAlwaysReturnZero() public {
2           // copy/paste of testDepositSwap because follows the same
                function pattern
3           vm.startPrank(liquidityProvider);
4           weth.approve(address(pool), 100e18);
5           poolToken.approve(address(pool), 100e18);
6           pool.deposit(100e18, 100e18, 100e18, uint64(block.timestamp));
7           vm.stopPrank();
8
9           vm.startPrank(user);
10          poolToken.approve(address(pool), 10e18);
11          uint256 expected = 9e18;
12
13          // get the returned value by 'swapExactInput'
14          uint256 expectedOutputAmount = pool.swapExactInput(
15              poolToken,
16              10e18,
```

```
17                 weth,
18                 expected,
19                 uint64(block.timestamp)
20             );
21             // assert the returned value its zero, no matter what we
                   swapped
22             assertEq(expectedOutputAmount, 0);
23         }
```

Recommended Mitigation:

1. If the returned value is not needed, consider to remove the returned value and harmonize the rest of the function.
2. If the returned value is needed, consider using a similar approach like the following example:

```
1
2      function swapExactInput(
3          IERC20 inputToken,
4          uint256 inputAmount,
5          IERC20 outputToken,
6          uint256 minOutputAmount,
7          uint64 deadline
8      )
9          public
10         revertIfZero(inputAmount)
11         revertIfDeadlinePassed(deadline)
12         returns (
13             uint256 output
14         )
15     {
16         uint256 inputReserves = inputToken.balanceOf(address(this));
17         uint256 outputReserves = outputToken.balanceOf(address(this));
18
19 -       uint256 outputAmount = getOutputAmountBasedOnInput(inputAmount,
       inputReserves, outputReserves);
20 +       output = getOutputAmountBasedOnInput(inputAmount, inputReserves
       , outputReserves);
21
22 -       if (outputAmount < minOutputAmount) {
23 -           revert TSwapPool__OutputTooLow(outputAmount,
       minOutputAmount);
24 -       }
25 +       if (output < minOutputAmount) {
26 +           revert TSwapPool__OutputTooLow(output, minOutputAmount);
27 +       }
28
29 -       _swap(inputToken, inputAmount, outputToken, outputAmount);
30 +       _swap(inputToken, inputAmount, outputToken, output);
31     }
```

Informational

[I-1] - At `PoolFactory::PoolFactory__PoolDoesNotExist` error is not used and should be removed.

Description: The error `PoolFactory__PoolDoesNotExist` is not used in the codebase. It should be removed.

Impact: Low.

Recommended Mitigation: Remove the `PoolFactory__PoolDoesNotExist` error from the codebase.

```
- error PoolFactory__PoolDoesNotExist(address tokenAddress);
```

[I-2] - `TSwapPool::Swap` event is missing `indexed` fields.

Description:

Index event fields make the field more quickly accessible to off-chain tools that parse events. However, note that each index field costs extra gas during emission, so it's not necessarily best to index the maximum allowed per event (threefields). Each event should use three indexed fields if there are three or more fields, and gas usage is not particularly of concern for the events in question. If there are fewer than three fields, all of the fields should be indexed.

The `TSwapPool::Swap` event should have at least 3 indexed parameters.

Impact: low.

Proof of Concept: (Proof of Code)

This is the actual code for the `Swap` event:

```
    event Swap(
        address indexed swapper,
        IERC20 tokenIn,
        uint256 amountTokenIn,
        IERC20 tokenOut,
        uint256 amountTokenOut
    );
```

Recommended Mitigation: Change the event parameters type to the following:

```
event Swap(
    address indexed swapper,
-   IERC20 tokenIn,
```

```
+   address indexed tokenIn,
    uint256 amountTokenIn,
-   IERC20 tokenOut,
+   address indexed tokenOut,
    uint256 amountTokenOut
);
```

[I-3] - `TSwapPool::constructor` is lacking zero address checks.

Description: At `TSwapPool::constructor` requires 2 inputed addresses `poolToken` & `wethToken` to initialize the contract with `i_wethToken` & `i_poolToken` values, but there is no zero address checks in the constructor, this might lead to an incorrect initialization of the contract.

Impact: HIGH.

Proof of Concept: (Proof of Code)

The following code is the actual codebase of the constructor for `TSwapPool`

```
    constructor(
        address poolToken,
        address wethToken,
        string memory liquidityTokenName,
        string memory liquidityTokenSymbol
    ) ERC20(liquidityTokenName, liquidityTokenSymbol) {
        i_wethToken = IERC20(wethToken);
        i_poolToken = IERC20(poolToken);
    }
```

Recommended Mitigation: Add a check for each of the inputed addresses.

```
+   error TSwapPool__ZeroAddressNotAllowed();

    constructor(
        address poolToken,
        address wethToken,
        string memory liquidityTokenName,
        string memory liquidityTokenSymbol
    ) ERC20(liquidityTokenName, liquidityTokenSymbol) {
+       if(poolToken == address(0) || wethToken == address(0)){
+           revert TSwapPool__ZeroAddressNotAllowed();
+       }
        i_wethToken = IERC20(wethToken);
        i_poolToken = IERC20(poolToken);
    }
```

[I-4] - `PoolFactory::createPool` at `liquidityTokenName` weird erc20 might not have the `.name()` function, requires a check.

Description:

Impact:

Proof of Concept: (Proof of Code)

Recommended Mitigation:

[I-5] - `PoolFactory::createPool` at `liquidityTokenSymbol` should be .symbol() instead of .name()

Description:

Impact:

Proof of Concept: (Proof of Code)

Recommended Mitigation: Change the following line:

```
    function createPool(address tokenAddress) external returns (address) {
        if (s_pools[tokenAddress] != address(0)) {
            revert PoolFactory__PoolAlreadyExists(tokenAddress);
        }
        string memory liquidityTokenName = string.concat("T-Swap ", IERC20(tokenAddress).name());
-       string memory liquidityTokenSymbol = string.concat("ts", IERC20(tokenAddress).name());
+       string memory liquidityTokenSymbol = string.concat("ts", IERC20(tokenAddress).symbol());
        TSwapPool tPool = new TSwapPool(tokenAddress, i_wethToken, liquidityTokenName, liquidityTokenSymbol);
        s_pools[tokenAddress] = address(tPool);
        s_tokens[address(tPool)] = tokenAddress;
        emit PoolCreated(tokenAddress, address(tPool));
        return address(tPool);
    }
```

[I-6] - `TSwapPool::MINIMUM_WETH_LIQUIDITY` is a constant, therefore not require to be emitted

Description: The `MINIMUM_WETH_LIQUIDITY` constant is used in the `deposit` function to check if the deposited WETH amount meets the minimum requirement. Since it is a constant value, it does not need to be emitted in the event logs.

Impact: Low

Proof of Concept: (Proof of Code)

This is the actual code for the function:

view code

```
    function deposit(
        uint256 wethToDeposit,
        uint256 minimumLiquidityTokensToMint,
        uint256 maximumPoolTokensToDeposit,
        uint64 deadline
    )
        external
        revertIfZero(wethToDeposit)
        returns (uint256 liquidityTokensToMint)
    {
        if (wethToDeposit < MINIMUM_WETH_LIQUIDITY) {
            revert TSwapPool__WethDepositAmountTooLow(
                MINIMUM_WETH_LIQUIDITY,
                wethToDeposit
            );
        }
```

Recommended Mitigation: Refactor the `TSwapPool__WethDepositAmountTooLow` error to match the following suggestion:

```
    error TSwapPool__WethDepositAmountTooLow(
-       uint256 minimumWethDeposit,
        uint256 wethToDeposit
    );
```

Harmonize the rest of the code to match the new error signature.

[I-7] - `TSwapPool::deposit` unused `poolTokenReserves` variable.

Description: `TSwapPool::deposit` declares an unused `poolTokenReserves` variable.

Impact: Low

Proof of Concept: (Proof of Code)

This is the actual code for the function:

view code

```
    function deposit(
        uint256 wethToDeposit,
        uint256 minimumLiquidityTokensToMint,
```

```
4           uint256 maximumPoolTokensToDeposit,
5           uint64 deadline
6       )
7           external
8           revertIfZero(wethToDeposit)
9           returns (uint256 liquidityTokensToMint)
10      {
11          if (wethToDeposit < MINIMUM_WETH_LIQUIDITY) {
12              revert TSwapPool__WethDepositAmountTooLow(
13                  MINIMUM_WETH_LIQUIDITY,
14                  wethToDeposit
15              );
16          }
17          if (totalLiquidityTokenSupply() > 0) {
18              uint256 wethReserves = i_wethToken.balanceOf(address(this))
                    ;
19  @>          uint256 poolTokenReserves = i_poolToken.balanceOf(address(
        this));
```

Recommended Mitigation: Remove the `poolTokenReserves` variable from the function, this will also help to save gas.

[I-8] - `TSwapPool::deposit` should follow CEI.

Description: `TSwapPool::deposit` should follow the Checks-Effects-Interactions (CEI) pattern to prevent reentrancy attacks.

Impact: Low

Proof of Concept: (Proof of Code)

This is the actual code for the function:

view code

```
1       function deposit(
2           uint256 wethToDeposit,
3           uint256 minimumLiquidityTokensToMint,
4           uint256 maximumPoolTokensToDeposit,
5           uint64 deadline
6       )
7           external
8           revertIfZero(wethToDeposit)
9           returns (uint256 liquidityTokensToMint)
10      {
11          if (wethToDeposit < MINIMUM_WETH_LIQUIDITY) {
12              revert TSwapPool__WethDepositAmountTooLow(
13                  MINIMUM_WETH_LIQUIDITY,
14                  wethToDeposit
```

```
15                 );
16             }
17             if (totalLiquidityTokenSupply() > 0) {
18                 // rest of the logic...
19             } else {
20                 // This will be the "initial" funding of the protocol. We
                       are starting from blank here!
21                 // We just have them send the tokens in, and we mint
                       liquidity tokens based on the weth
22                 _addLiquidityMintAndTransfer(
23                     wethToDeposit,
24                     maximumPoolTokensToDeposit,
25                     wethToDeposit
26                 );
27 @>             liquidityTokensToMint = wethToDeposit;
28             }
29         }
```

Recommended Mitigation: Move the `liquidityTokensToMint` assignment before the `_addLiquidityMintAndTransfer` call to follow the CEI pattern.

```
1              } else {
2  +               liquidityTokensToMint = wethToDeposit;
3                  // This will be the "initial" funding of the protocol. We
                       are starting from blank here!
4                  // We just have them send the tokens in, and we mint
                       liquidity tokens based on the weth
5                  _addLiquidityMintAndTransfer(
6                      wethToDeposit,
7                      maximumPoolTokensToDeposit,
8                      wethToDeposit
9                  );
10 -               liquidityTokensToMint = wethToDeposit;
11             }
```

[I-9] - Use constants instead of magic numbers

Description: In programming, magic numbers refers to the use of unexplained numerical or string values directly in code, without any clear indication of their purpose or origin. The use of magic numbers can lead to confusion and make your code more difficult to understand, maintain, and update.

To improve the readability and maintainability of your smart contracts, it is recommended to avoid using magic numbers and instead use named constants or variables to represent these values. By doing so, you provide clear context for the values, making it easier for developers to understand their purpose and significance.

The functions `TSwapPool::getOutputAmountBasedOnInput` & `TSwapPool::getInputAmountBasedO`

use magic numbers in their calculations, making the code less readable and maintainable.

Impact: Low

Proof of Concept: (Proof of Code)

This is the actual codebase used on `TSwapPool::getOutputAmountBasedOnInput` & `TSwapPool::getInputAmountBasedOnOutput` functions.

view code

```
1      function getOutputAmountBasedOnInput(
2          uint256 inputAmount,
3          uint256 inputReserves,
4          uint256 outputReserves
5      )
6          public
7          pure
8          revertIfZero(inputAmount)
9          revertIfZero(outputReserves)
10         returns (uint256 outputAmount)
11     {
12 @>       uint256 inputAmountMinusFee = inputAmount * 997;
13         uint256 numerator = inputAmountMinusFee * outputReserves;
14 @>       uint256 denominator = (inputReserves * 1000) +
       inputAmountMinusFee;
15         return numerator / denominator;
16     }
17
18     function getInputAmountBasedOnOutput(
19         uint256 outputAmount,
20         uint256 inputReserves,
21         uint256 outputReserves
22     )
23         public
24         pure
25         revertIfZero(outputAmount)
26         revertIfZero(outputReserves)
27         returns (uint256 inputAmount)
28     {
29         return
30 @>          ((inputReserves * outputAmount) * 10000) /
31 @>          ((outputReserves - outputAmount) * 997);
32     }
```

Recommended Mitigation: To improve code maintainability, readability, and reduce the risk of potential errors, it is recommended to replace magic numbers with well-defined constants. By using constants, developers can provide clear and descriptive names for specific values, making the code easier to understand and maintain. Additionally, updating the values becomes more straightforward, as changes can be made in a single location, reducing the risk of errors and inconsistencies. For large

numbers, consider using scientific notation (e.g., 1e4).

Suggestions:

```
+    uint256 constant PRECISION = 1000;
+    uint256 constant MIN_INPUT_FEE = 997;

     function getOutputAmountBasedOnInput(
         uint256 inputAmount,
         uint256 inputReserves,
         uint256 outputReserves
     )
         public
         pure
         revertIfZero(inputAmount)
         revertIfZero(outputReserves)
         returns (uint256 outputAmount)
     {
-        uint256 inputAmountMinusFee = inputAmount * 997;
+        uint256 inputAmountMinusFee = inputAmount * MIN_INPUT_FEE;
         uint256 numerator = inputAmountMinusFee * outputReserves;
-        uint256 denominator = (inputReserves * 1000) + inputAmountMinusFee;
+        uint256 denominator = (inputReserves * PRECISION) + inputAmountMinusFee;
         return numerator / denominator;
     }

     function getInputAmountBasedOnOutput(
         uint256 outputAmount,
         uint256 inputReserves,
         uint256 outputReserves
     )
         public
         pure
         revertIfZero(outputAmount)
         revertIfZero(outputReserves)
         returns (uint256 inputAmount)
     {
         return
-           ((inputReserves * outputAmount) * 10000) /
-           ((outputReserves - outputAmount) * 997);
+           ((inputReserves * outputAmount) * PRECISION) /
+           ((outputReserves - outputAmount) * MIN_INPUT_FEE);
     }
```

[I-10] - `TSwapPool::swapExactInput` function is missing NatSpec Comments.

Description: All the contracts in the code-base are missing or have incomplete code documentation, which affects the understandability, auditability, and usability of the code. Solidity contracts can use a special form of comments to provide rich documentation for functions, return variables, parameters, etc. This special form is named the Ethereum Natural Language Specification Format (NatSpec).

Impact: Low.

Proof of Concept: The actual code does not have any NatSpec comments, which makes it difficult for developers to understand the purpose and usage of the `swapExactInput` function.

Recommended Mitigation: Consider adding in full NatSpec comments for all functions to have complete code documentation for future use.

[I-11] - `TSwapPool::swapExactInput` function can be made external.

Description: The `TSwapPool::swapExactInput` function is defined as public. If a function is marked public but is not used internally, consider marking it as `external`.

Impact: Low.

Proof of Concept: The actual implementation of the `TSwapPool::swapExactInput` function is marked as `public`.

Recommended Mitigation: We recommend making the `TSwapPool::swapExactInput` function external.

[I-12] - `TSwapPool::totalLiquidityTokenSupply` function can be made external.

Description: The `TSwapPool::totalLiquidityTokenSupply` function is defined as public. If a function is marked public but is not used internally, consider marking it as `external`.

Impact: Low.

Proof of Concept: The actual implementation of the `TSwapPool::totalLiquidityTokenSupply` function is marked as `public`.

Recommended Mitigation: We recommend making the `TSwapPool::totalLiquidityTokenSupply` function external.

Gas